

Design and Evaluation of a Reproducible Docker-Based Network Digital Twin for Controlled Network Experimentation

Anonymous authors

Paper under double-blind review

Abstract

Network digital twins promise safer and more repeatable network experimentation, but practical evidence is often fragmented across architecture proposals, emulators, topology studies, and automated control demonstrations. This dissertation designs and evaluates a scenario-driven network digital twin that uses Docker containers, deterministic routing, Linux traffic control, and a common evidence pipeline for controlled experiments. The evaluation covers bandwidth, configured one-way delay, configured packet loss, direct and multi-router topologies, a 50-node/88-link Germany50 topology, validated AI-assisted scenario generation, and real-Docker closed-loop control.

Repeated experiments show that the platform applies measurable network conditions while retaining raw and structured evidence. At configured bandwidths of 50 and 100 Mbps, five valid runs produced mean throughputs of 41.500 and 78.780 Mbps, respectively; a separate historical 20-Mbps record measured 19.2 Mbps and is not treated as a repeated cohort. Five-run delay conditions of 0, 10, 30, and 50 ms one way produced mean measured RTTs of 0.105, 27.008, 80.303, and 133.686 ms. Configured one-way loss of 0, 1, 3, and 5% produced mean end-to-end ping loss of 0, 3, 6, and 10%. A five-run dual-router benchmark averaged 19.220 Mbps. The complete Germany50 topology was instantiated, its 4,224-entry route plan was validated only in dry-run mode, and real traffic was measured on three representative paths rather than all pairs. The official OpenAI attempt returned HTTP 429, while a separate Qwen-compatible path succeeded after validation. Across 80 valid real-Docker RL episodes, the threshold heuristic materially outperformed the evaluated Q-learning configuration.

The results support a reproducible, extensible workflow within the evaluated configurations, while exposing limits in Docker-host fidelity, large-topology traffic coverage, provider availability, evidence preservation, and learning-based control performance.

1 Introduction

Modern networks are increasingly expected to support rapid configuration changes, stringent service requirements, and automated control. Testing these changes directly on operational infrastructure is costly and can be unsafe. A network digital twin (NDT) addresses this problem by providing a digital representation in which network state, models, and control interfaces can be used to evaluate behaviour before or alongside physical deployment. Current reference work describes NDTs through data, models, interfaces, and mappings, and presents closed-loop management as a central objective (Zhou et al., 2026; International Telecommunication Union, 2022). Data-driven NDT research further emphasizes fidelity, efficiency, and flexibility in network performance modelling (Almasan et al., 2022; Hui et al., 2023).

The practical challenge is not simply to instantiate a topology. A useful experimental twin must transform a scenario into a controlled execution, verify routing and impairment configuration, collect measurements, and retain enough provenance to support later analysis. Lightweight emulation established that real network applications and protocol stacks can be exercised on a single host (Lantz et al., 2010), while container-based emulation showed how runnable specifications and retained artifacts can improve reproducibility (Handigol

et al., 2012). However, architecture, emulation, topology scaling, AI-assisted configuration, and learning-based control are commonly evaluated as separate concerns. This fragmentation makes it difficult to assess whether they can share one bounded execution and evidence workflow without overstating fidelity or automation.

This dissertation addresses that integration problem through a scenario-driven Docker NDT. JSON scenarios are validated before execution; Docker containers and networks implement endpoint and router roles; deterministic forwarding and routes establish reachability; Linux `tc`, `netem`, and token-bucket filtering impose bandwidth, delay, and loss; and ping and iperf3 produce measurements that are retained as structured evidence. The same guarded execution environment is extended to a realistic Germany50-derived topology, validated AI-assisted scenario generation, a real-Docker control loop, and a deliberately bounded interactive Dashboard.

1.1 Research gap and questions

Existing work separately provides NDT architectures, reproducible/container-based emulation, realistic topology datasets, AI-assisted networking, and digital-twin-assisted reinforcement learning. The research gap considered here is the integrated, quantitative evaluation of these elements in a common scenario-driven Docker workflow. The work does not claim to be the first network emulator, NDT, AI networking system, or NDT/RL system. Instead, it asks whether a compact implementation can enforce controlled conditions, scale its topology and routing mechanisms, support guarded automation, and preserve claim-ready evidence.

The main research question is:

How effectively can a Docker-based network digital twin enforce and measure controlled network conditions while remaining reproducible and extensible across increasingly complex topologies and automated control workflows?

It is decomposed into three questions:

- RQ1:** How closely and consistently do measured network behaviours follow configured bandwidth, latency, and packet-loss conditions across repeated experiments?
- RQ2:** How does the framework extend from simple Docker topologies to multi-router and larger network topologies while preserving validated routing and measurable real traffic?
- RQ3:** Can validated AI-assisted scenario generation and closed-loop learning-based control operate on the same digital-twin execution environment, and what limitations emerge in doing so?

1.2 Contributions

The dissertation makes three bounded contributions. First, it presents a reproducible, scenario-driven Docker NDT framework that joins validation, topology construction, routing, impairment, measurement, cleanup, and evidence retention. Second, it provides a quantitative evaluation of controlled bandwidth, one-way delay, packet loss, multi-router execution, and Germany50 topology scaling. Third, it evaluates guarded AI-assisted scenario generation and a real-Docker closed-loop controller, including the negative result that the evaluated threshold heuristic outperformed Q-learning. These contributions are limited to the retained configurations: Germany50 real traffic covers three representative paths, the full route plan is dry-run evidence, provider outcomes remain separate, and the Dashboard is a supporting small-topology interface.

The remainder of this dissertation reviews related work, describes the system and experimental methods, reports the retained results, discusses the research questions and threats to validity, and concludes with the scope of the supported claims.

2 Background and Related Work

2.1 Network digital twins

The NDT concept combines a representation of a real network with data, models, mappings, and interfaces that support analysis and control. The IRTF architecture draft organizes these elements into a reference architecture and use cases, but remains a work in progress rather than a formal standard (Zhou et al., 2026). ITU-T Recommendation Y.3090 specifies requirements and a three-layer, three-domain, double-closed-loop architecture (International Telecommunication Union, 2022). These sources provide an architectural vocabulary; neither establishes the empirical fidelity of a particular container implementation.

Almasan et al. position the NDT as a real-time, data-driven model and demonstrate its use for QoS-aware routing optimization (Almasan et al., 2022). Hui et al. examine NDT performance modelling through the competing requirements of fidelity, efficiency, and flexibility (Hui et al., 2023). Their model-centric perspective is complementary to this dissertation. Here, the twin is not presented as a learned surrogate of an operational network; it is an executable, controlled environment whose claims depend on measured container traffic and retained evidence.

2.2 Container-based network emulation and reproducibility

Mininet demonstrated that operating-system virtualization can place hosts, switches, links, and real application code on one machine, enabling rapid SDN prototyping (Lantz et al., 2010). Handigol et al. subsequently used high-fidelity container-based emulation to reproduce published network experiments, showing the importance of runnable specifications, isolation, and complete experimental artifacts (Handigol et al., 2012). MeDICINE extended this line toward containerized, production-oriented network functions in multi-PoP environments and interfaces to external orchestration systems (Peuster et al., 2016). Docker itself packages processes and dependencies using kernel namespaces and control groups, improving deployment consistency without eliminating shared-kernel interference (Merkel, 2014).

This literature establishes both the opportunity and the caution for the present work. Containers can run real stacks and tools, but reproducibility is not guaranteed by packaging alone. Host scheduling, kernel state, image versions, traffic-control placement, and missing artifacts can all alter what can later be claimed. The framework therefore treats scenario validation, route and qdisc checks, structured outputs, failure retention, and scoped cleanup as part of the research method rather than incidental implementation details.

2.3 Controlled impairment and topology scaling

Linux `netem` supports delay, loss, corruption, duplication, reordering, and rate-related effects. Its documentation also warns that timer granularity, TCP small queues, and queue placement can influence observations (iproute2 Project, n.d.). Consequently, a configured one-way delay or loss probability is not itself a measured end-to-end result. This dissertation reports ping RTT and end-to-end ping loss as separate constructs and uses `iperf3` throughput to assess bandwidth enforcement.

Realistic topology data provide a different scaling dimension. The Internet Topology Zoo curates operator-supplied network graphs and metadata, enabling structural studies that are more realistic than synthetic graph families (Knight et al., 2011). Germany50 supplies 50 nodes and 88 links for this evaluation. A topology graph alone, however, does not demonstrate route installation or traffic coverage. The present work therefore distinguishes topology instantiation, complete route-plan dry-run validation, and real traffic on selected representative paths.

2.4 AI-assisted networking

Large language models (LLMs) are being investigated for configuration, diagnosis, and network management. Lira et al. propose LLM-NetCFG, in which natural-language intents are translated into configurations and then verified before device application (Lira et al., 2025). Liu et al. synthesize LLM-for-networking work into a broader workflow and identify reliability, domain knowledge, security, and evaluation challenges (Liu

et al., 2025). These concerns motivate a guarded design: generated content is treated as untrusted data, subjected to schema and semantic validation, projected into an allowlisted scenario, and only then permitted to reach controlled execution. Successful use of an OpenAI-compatible provider does not establish success of the official OpenAI service.

2.5 Reinforcement learning for network control

RL offers a way to learn repeated decisions from observed state and reward. DeepRM demonstrated that resource-management decisions can be formulated as a learning problem and compared with heuristic schedulers (Mao et al., 2016). In an NDT context, Güemes-Palau et al. use evolutionary strategies to accelerate DRL training for routing optimization (Güemes-Palau et al., 2022), while Cheng et al. propose digital-twin-assisted RL to improve exploration safety, convergence, and resource-management performance (Cheng et al., 2025). These studies justify evaluating a learnable controller, but they do not imply that RL should outperform a transparent heuristic in every low-dimensional environment.

2.6 Gap synthesis

The related work provides mature components but no automatic guarantee that their integration will be reproducible or empirically persuasive. Architecture documents do not validate container fidelity; emulators do not by themselves supply a claim/evidence discipline; topology datasets do not prove route or traffic coverage; LLM generation requires validation and provider-specific reporting; and RL must be compared against credible baselines. This dissertation therefore evaluates a common pipeline across impairment, topology, AI, and control tasks, and treats negative results and missing raw evidence as part of the contribution rather than hiding them.

3 System Design and Methodology

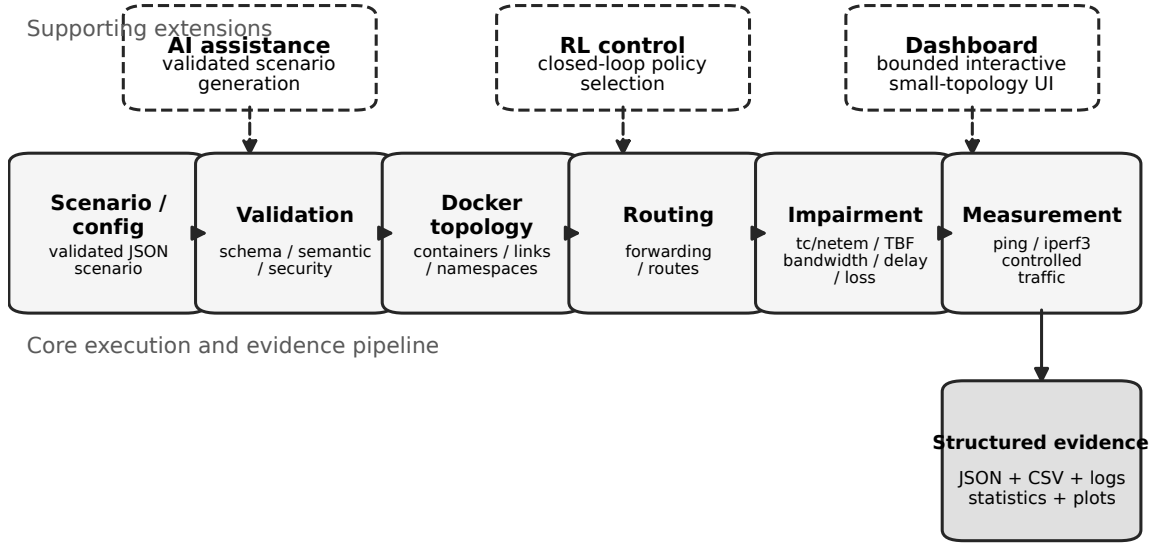
3.1 Objectives and scope

The system was designed around five objectives: (i) scenarios should be explicit and machine-validated; (ii) execution should use real container network stacks and standard Linux tools; (iii) routing and impairment state should be checked rather than assumed; (iv) each run should produce structured, traceable evidence; and (v) extensions should reuse the same bounded execution path. The system is an experimental NDT, not a synchronized replica of a production network. Its supported claims concern controlled behaviour and workflow reproducibility within evaluated configurations.

Figure 1 summarizes the execution path. A scenario/configuration enters validation before any topology is built. The validated representation drives Docker topology creation, routing and forwarding, impairment configuration, and measurement. Results are serialized as JSON/CSV and accompanied where available by logs, summaries, statistics, and plots. Cleanup is scoped to project-owned resources so that a failed run does not justify broad container or network deletion.

3.2 Scenario representation and validation

Scenarios describe topology identity, node roles, links, addressing, impairments, and traffic endpoints. Small experiments use allowlisted direct, routed, or two-router forms; the generic topology path supports larger graphs. Validation is layered. Schema checks reject malformed structures and unexpected fields. Semantic checks enforce unique identifiers, valid endpoints, bounded numeric parameters, and consistent links. For AI-originated content, prompt constraints and a projection stage further remove operational freedom: generated output cannot select arbitrary commands, images, credentials, or host paths. This makes validation a security boundary as well as a correctness check.



Arrows show validated control flow; evidence artifacts are retained after bounded cleanup.

Figure 1: Overall architecture. Scenario validation, Docker execution, routing, impairment, measurement, and evidence retention form the core pipeline. AI assistance, RL control, and the Dashboard are bounded extensions rather than independent core systems.

3.3 Docker topology and routing

Nodes are implemented as Docker containers attached to deterministic Docker networks. Routers enable forwarding; endpoints and routers receive addresses derived from the scenario. Direct scenarios require no intermediate route, while routed and multi-router scenarios install explicit paths and verify expected reachability. The generic topology path constructs a network per link and provisions routes required by the selected traffic pair. Route checks are retained with run evidence, avoiding the inference that container creation alone implies end-to-end connectivity.

Germany50 extends this mechanism using imported Topology Zoo data. The normalized scenario contains 50 router nodes and 88 links with deterministic subnets. Two routing modes were retained. Selected mode installs the routes needed for the three representative endpoint pairs. Full mode generates the theoretical 4,224-entry plan, but the retained full-mode evidence is a dry-run validation only. This separation is fundamental: plan generation, route installation, and traffic exercise are distinct claims.

3.4 Impairment and measurement pipeline

Linux traffic control implements the configured conditions. Token-bucket filtering (TBF) constrains bandwidth, while `netem` introduces one-way delay and packet loss. The framework records qdisc state where available and then executes ping and reverse TCP iperf3 traffic. Ping provides sent/received counts, loss, and RTT; iperf3 provides measured throughput. Configured one-way delay is therefore never relabelled as measured latency, and configured one-way loss is not equated with measured end-to-end loss.

Each run writes its normalized scenario and metrics into a run-specific directory. Batch runners add manifests, per-run rows, summaries, environment records, and cleanup outcomes. Failed or retried attempts remain visible. The dissertation inventory identifies which values were already repository-reported and which statistics were recomputed during a read-only audit.

3.5 Automation extensions

AI-assisted generation. The AI path requests a structured scenario, validates it, projects it into the executable representation, and permits only dry-run or controlled execution. Provider identity is retained separately from validation outcome. The official OpenAI call and the OpenAI-compatible Qwen call therefore constitute different evidence items.

Closed-loop control. The RL extension uses a minimal dual-path Docker topology. At each episode the controller observes measured network state, selects a path, verifies routing, runs ping and iperf3, computes reward, and checkpoints the outcome. Q-learning is compared with a threshold heuristic and two fixed-path policies under the same ordered two-phase schedule. The design tests whether learning can operate on real Docker measurements; it does not assume that learning is the best policy.

Dashboard. The zero-dependency interactive server exposes allowlisted small-topology experiments on loopback. It supports validation, dry-run lifecycle, a confirmed direct real-Docker path, restricted downloads, and scoped cleanup. Germany50 and RL results remain read-only, and arbitrary Docker commands are not accepted. Because raw UI-path and regression/security logs were not retained, Dashboard evidence is used conservatively.

3.6 Reproducibility and security

Reproducibility is supported by versioned scenarios, runner scripts, per-run metrics, summaries, figures, failure records, and documentation. It does not mean that every environment will produce identical timing: Docker shares the host kernel and traffic-control timing is host-sensitive. Security measures include input allowlists, semantic bounds, separation of provider responses from execution, local-only Dashboard operation, argument-list Docker invocation, and project-scoped resource names. The retained repository does not contain the claimed raw fresh-clone, 69-test, secret-scan, or security-scan logs; this absence is reported rather than reconstructed.

4 Experimental Methodology

4.1 Environment, metrics, and statistics

The retained experiments ran in the project’s Docker/WSL environment using project container images, Python runners, Linux traffic control, ping, and iperf3. Exact host hardware utilization was not retained consistently, so hardware-level performance generalization is avoided. Every reported result is tied to a run directory or repository summary in the evidence inventory.

The primary metrics are TCP throughput in Mbps, ping RTT in milliseconds, end-to-end ping packet loss in percent, and controller reward. For complete per-run cohorts, the audit uses the sample standard deviation and a two-sided Student’s t 95% confidence interval,

$$\bar{x} \pm t_{0.975, n-1} \frac{s}{\sqrt{n}}.$$

Values already present in repository summaries are labelled repository-reported (RR); statistics recomputed without changing the evidence repository are labelled audit-computed (AC). No missing measurements are imputed.

4.2 Controlled impairment experiments

The bandwidth evidence has deliberately unequal provenance. The direct 20-Mbps result is one retained historical client–router–server measurement using server-side TBF. It is reported as $n = 1$. The 50- and 100-Mbps results are later supplementary replications of the direct two-node, server-side TBF, five-second reverse-TCP iperf3 method. The 50-Mbps cohort contains five valid runs. The 100-Mbps cohort contains five valid measurements from six attempts; the failed attempt and explicit retry remain in the evidence.

Table 1: Experimental matrix. Runs count retained attempts; valid runs are the observations included in the reported result.

Experiment	Configuration	Runs / valid	Primary metric
Bandwidth	Direct: 20, 50, 100 Mbps	1/1; 5/5; 6/5	Throughput (Mbps)
Latency	0, 10, 30, 50 ms configured one-way delay	5/5 each	Ping RTT (ms)
Packet loss	0, 1, 3, 5% configured one-way loss; 100 pings/run	5/5 each	End-to-end loss (%)
Dual-router	Client-router-router-server; 20 Mbps; zero delay/loss	5/5	Throughput (Mbps)
Germany50 paths	Extracted shortest, median, longest paths; 20 Mbps, 10 ms/link	3/3 each	RTT and throughput
Germany50 full	50 nodes, 88 links; selected routes and full-plan dry-run	9/9 + dry-run	Scope and reachability
AI generation	Official OpenAI attempt; separate Qwen-compatible path	1/0; 1/1	Validation/API outcome
RL control	Q-learning, threshold heuristic, fixed A, fixed B	20/20 each	Reward; RTT/loss/rate
Dashboard	Documented direct UI-path smoke	1 documented	RTT and throughput

Latency used a 20-Mbps routed topology at configured one-way delays of 0, 10, 30, and 50 ms. Each condition has five valid runs and three ping packets per run. The dependent variable is measured RTT, not one-way latency. Packet-loss experiments configured one-way loss of 0, 1, 3, and 5%, with five runs and 100 ping packets per run. Sent and received counts determine measured end-to-end loss. The dual-router benchmark used client-router1-router2-server, zero configured delay/loss, and five runs; it is analyzed separately from direct bandwidth evidence.

4.3 Topology, AI, control, and interface experiments

Three Germany50-derived paths were fixed before execution: Aachen-Koeln (one backbone hop), Erfurt-Wuerzburg-Augsburg-Muenchen-Passau (four), and Oldenburg-Bremen-Hannover-Braunschweig-Kassel-Erfurt-Wuerzburg-Augsburg-Muenchen-Passau (nine). Extracted-path experiments used 20 Mbps, 10-ms one-way delay, and 0% loss on each link, with three runs per path. A separate full-topology evaluation instantiated all 50 nodes and 88 links. Selected routing installed 20 routes for three endpoint pairs and produced nine real traffic measurements. Full routing generated and dry-run validated 4,224 entries but did not install or exercise them as all-pairs traffic.

AI validation distinguishes the official OpenAI attempt from a separate OpenAI-compatible Qwen path. The official request produced HTTP 429 `insufficient_quota`. The Qwen-compatible path produced a six-node scenario that passed schema, semantic, prompt-constraint, projection, dry-run, and controlled real-Docker validation.

The RL comparison contains 80 valid real-Docker episodes: 20 each for Q-learning, threshold heuristic, fixed A, and fixed B. All policies experienced the same ordered two-phase schedule. The recorded Q-learning configuration used seed 20260722, learning rate $\alpha = 0.35$, discount $\gamma = 0.85$, and $\epsilon = 0.18$. Two failed trials were retained as failed/retried checkpoints and are not counted among the 80 valid rows. The Dashboard result is a documented single direct UI-path smoke, not a repeated experiment; absence of its raw validation logs determines its documented-only status.

5 Results

5.1 Controlled impairments

Figure 2 and Table 2 report the main impairment results. The direct historical 20-Mbps record measured 19.2 Mbps, a -4.00% deviation, but supplies no SD or confidence interval. The supplementary 50-Mbps cohort measured 41.500 Mbps (SD 3.362, 95% CI ± 4.175 , range 36.9–45.8), a -17.00% deviation. At 100 Mbps the five valid measurements averaged 78.780 Mbps (SD 11.684, 95% CI ± 14.508 , range 58.4–87.3), a -21.22% deviation. These 50/100-Mbps statistics are repository-reported from the retained supplementary cohort. The dual-router 20-Mbps benchmark is excluded from this direct-topology panel.

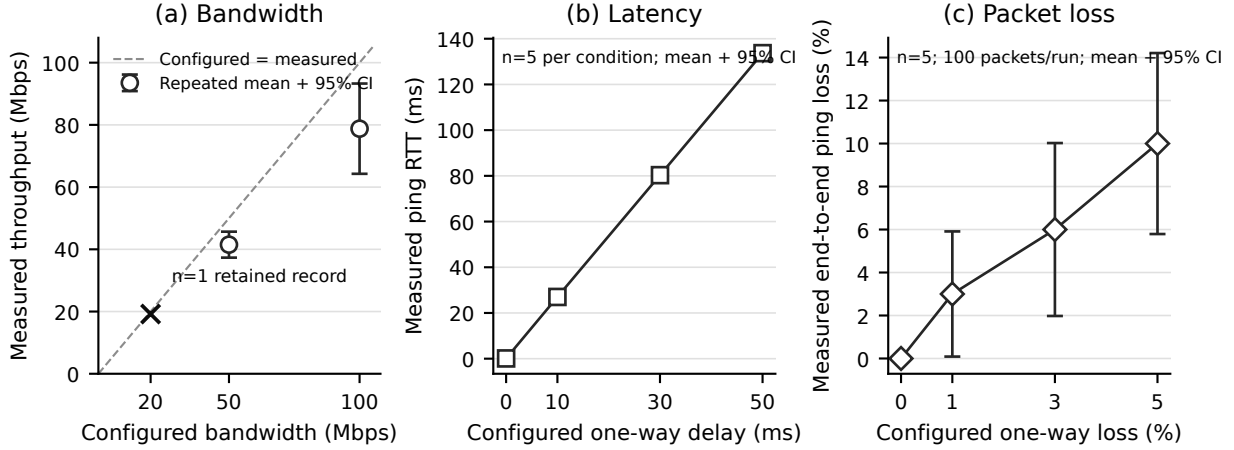


Figure 2: Main impairment results. Bandwidth distinguishes the single historical 20-Mbps record from replicated 50/100-Mbps cohorts. Latency plots configured one-way delay against measured ping RTT. Packet loss plots configured one-way loss against measured end-to-end ping loss. Error bars are shown only for repeated cohorts and represent 95% confidence intervals.

Table 2: Main impairment results. CI is the retained or audit-computed 95% confidence-interval half-width. RR denotes repository-reported and AC denotes audit-computed from complete per-run evidence.

Configuration	n	Mean	SD	95% CI	Min	Max	Deviation / provenance
BW 20 Mbps*	1	19.200	—	—	19.200	19.200	—4.00%; RR
BW 50 Mbps	5	41.500	3.362	± 4.175	36.900	45.800	—17.00%; RR
BW 100 Mbps	5	78.780	11.684	± 14.508	58.400	87.300	—21.22%; RR
Delay 0 ms	5	0.105	0.019	± 0.024	0.080	0.123	RTT; RR/AC
Delay 10 ms	5	27.008	0.056	± 0.070	26.943	27.089	RTT; RR/AC
Delay 30 ms	5	80.303	0.057	± 0.071	80.249	80.393	RTT; RR/AC
Delay 50 ms	5	133.686	0.037	± 0.046	133.637	133.723	RTT; RR/AC
Loss 0%	5	0.000	0.000	± 0.000	0	0	end-to-end; RR/AC
Loss 1%	5	3.000	2.345	± 2.911	1	7	end-to-end; RR/AC
Loss 3%	5	6.000	3.240	± 4.023	2	9	end-to-end; RR/AC
Loss 5%	5	10.000	3.391	± 4.210	5	14	end-to-end; RR/AC

*The direct 20-Mbps value is one historical retained record, not a replicated cohort. Bandwidth units are Mbps; delay means are measured RTT in ms; loss means are measured end-to-end ping loss in percent. Configured one-way delay/loss and measured end-to-end quantities are not treated as direct errors.

Mean RTT increased from 0.105 ms at 0-ms configured one-way delay to 27.008, 80.303, and 133.686 ms at 10, 30, and 50 ms. The corresponding AC sample SDs were 0.019, 0.056, 0.057, and 0.037 ms, with CI half-widths 0.024, 0.070, 0.071, and 0.046 ms. The means are RR; SD, CI, minimum, and maximum values in Table 2 are AC. These observations are RTTs and are not reported as measured one-way latency.

Configured one-way loss of 0, 1, 3, and 5% produced RR mean end-to-end ping loss of 0, 3, 6, and 10%. Across five runs per condition, the AC SDs were 0, 2.345, 3.240, and 3.391 percentage points; confidence intervals widened accordingly. Aggregate packet counts were 500/500, 485/500, 470/500, and 450/500 received/sent. Because impairment configuration and ping observation cover different directions and constructs, no direct relative-error statistic is assigned.

The separate dual-router benchmark completed five valid runs with a mean throughput of 19.220 Mbps, SD 0.045, 95% CI ± 0.056 , and range 19.2–19.3 Mbps. The mean is RR; dispersion and range are AC.

5.2 Topology scaling and Germany50

Figure 3 separates simple execution, the dual-router benchmark, and full Germany50 scope. In the controlled extracted-path experiments, the one-hop shortest path produced mean RTT 41.196 ms (AC SD 0.259, CI ± 0.642) and mean throughput 18.8 Mbps. The four-hop median path produced 104.623 ms (SD 0.845, CI

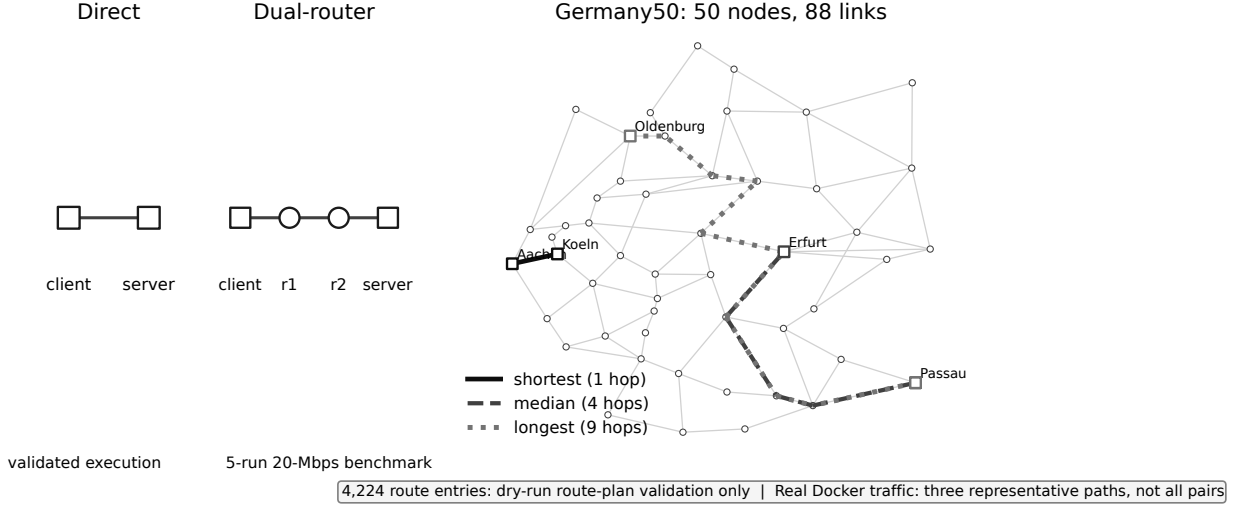


Figure 3: Topology scaling from direct and dual-router configurations to the real 50-node/88-link Germany50 graph. Highlighted paths are the shortest, median, and longest representative paths used for real traffic. The 4,224-entry full route plan was validated in dry-run mode only; the figure does not represent all-pairs traffic.

Table 3: Extended validation summary and evidence boundaries.

Component	Retained evidence	Result	Required interpretation
Dual-router	Five real-Docker runs	19.220 Mbps mean, SD 0.045, 95% CI ± 0.056	Separate 20-Mbps multi-router benchmark; not part of the direct cohort.
Germany50	Full topology, route summaries, nine selected-path measurements	50 nodes, 88 links; 4,224-route plan passed dry-run; shortest/median/longest carried real traffic	Dry-run plan is not route installation or all-pairs real traffic.
AI	Sanitized official-provider result; compatible-provider scenario and run	Official OpenAI: HTTP 429 insufficient quota. Qwen-compatible: validated success	Provider outcomes are separate; Qwen success is not official OpenAI success.
RL	Per-episode CSV and policy summary	80 valid real-Docker episodes; heuristic reward 17.116 vs Q-learning 4.143	The threshold heuristic materially outperformed the evaluated Q-learning setup.
Dashboard	Validation document and implementation	Documented direct smoke: 0.118-ms mean RTT and 19.4 Mbps	Raw UI, 69-test, security, secret-scan, and fresh-clone logs were not retained.

± 2.098) and 17.8 Mbps. The nine-hop longest path produced 214.425 ms (SD 0.441, CI ± 1.095) and 15.7 Mbps. Relative to 20 Mbps, the reported throughput deviations were -6.0% , -11.0% , and -21.5% .

The full topology preflight recorded 50 nodes, 88 networks, and a theoretical 4,224-route plan. Selected mode installed 20 routes and completed three ping/iperf3 measurements for each representative pair (nine total). The full 4,224-entry plan separately returned dry-run success. No retained evidence shows that all entries were installed or that all node pairs exchanged real traffic.

5.3 AI, closed-loop control, and Dashboard

Table 3 summarizes the extensions. The official OpenAI attempt returned HTTP 429 with `insufficient_quota`; it generated no successful official scenario. The separate Qwen-compatible request produced a validated scenario and a single controlled real-Docker result of 82.194-ms RTT, 18.3-Mbps throughput, and 0% measured loss. No SD or CI is available for this single validation.

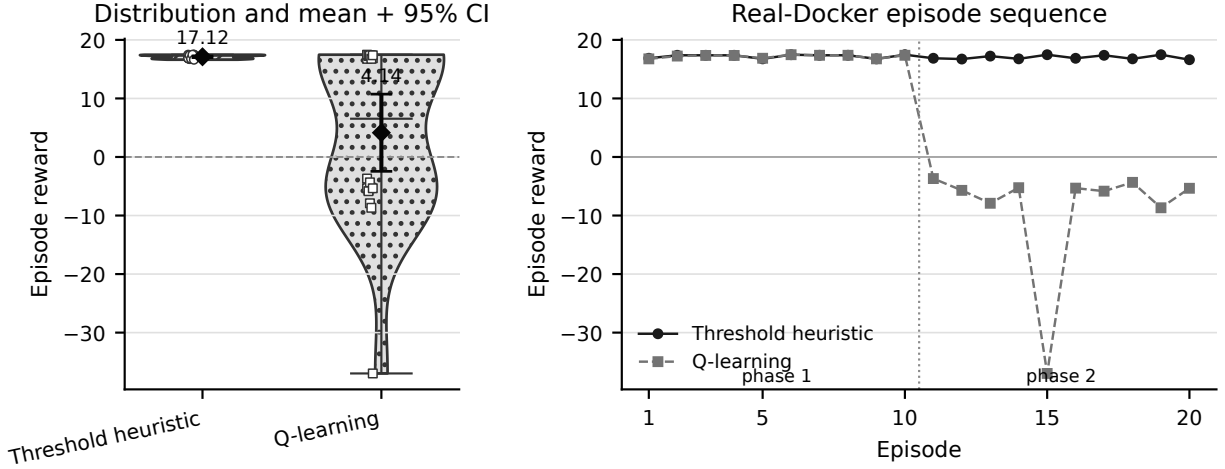


Figure 4: Reward comparison for the 20 valid real-Docker episodes of each displayed policy. The complete retained supplement contains 80 valid episodes across Q-learning, threshold heuristic, fixed A, and fixed B. Both the distribution and episode sequence show that the threshold heuristic materially outperformed Q-learning under the evaluated setup.

Across the balanced RL supplement, the threshold heuristic achieved RR mean reward 17.116 (SD 0.325, 95% CI ± 0.142), mean RTT 30.189 ms, 0% mean loss, and 18.625-Mbps mean throughput. Q-learning achieved mean reward 4.143 (SD 15.045, CI ± 6.594), mean RTT 89.225 ms, 2.5% loss, and 9.855-Mbps throughput. AC reward ranges were 16.615–17.471 for the heuristic and -36.975 –17.472 for Q-learning. Q-learning selected path A in all 20 episodes and made no switch; the heuristic selected A for ten episodes, B for ten, and switched once at the phase boundary.

The Dashboard validation document records direct run 3cada000bb4d: three of three ping replies, RTT min/mean/max 0.101/0.118/0.143 ms, 19.4-Mbps throughput, HTTP 200 for JSON/CSV/log/ZIP downloads, and cleanup pass. The raw UI-run, 69-test, security, secret-scan, and fresh-clone logs are not retained, so this result remains documented-only.

6 Discussion

6.1 RQ1: controllability and repeatability

The results show that the framework translates configured conditions into distinct and repeatable measured regimes, but not that measured values equal their configuration. Bandwidth remained below the TBF setting, with larger deviations and variability at 50 and 100 Mbps than in the single 20-Mbps record. This is consistent with throughput being an end-to-end TCP outcome rather than a direct reading of the configured token rate. Protocol overhead, TCP dynamics, container scheduling, virtual interfaces, and host load can all contribute. The single historical 20-Mbps record cannot establish repeatability and is therefore supporting evidence only.

Latency provides the clearest repeated trend. Within each configured condition, RTT dispersion was small, and mean RTT increased monotonically with configured one-way delay. Nevertheless, the measured RTT includes the complete end-to-end path and both directions, while the configuration is applied one way to particular interfaces. Treating the two quantities as identical would be a construct error. The packet-loss matrix similarly distinguishes configured one-way probability from end-to-end ping loss. Measured means exceeded the configured percentages at non-zero settings and confidence intervals were broad with only five runs, so the appropriate conclusion is controlled differentiation rather than exact calibration.

Together, these results answer RQ1 positively but conditionally: the NDT enforces stable, distinguishable impairment regimes and retains sufficient per-run evidence for repeated analysis, while accuracy depends on metric semantics and host-level effects. A strength of the workflow is that this difference is visible rather than hidden behind configuration values.

6.2 RQ2: topology extension and routing

The dual-router benchmark demonstrates that explicit forwarding and static routing preserve a near-20-Mbps flow across two routers. Germany50 then separates three forms of scaling evidence. First, extracted paths show that the same scenario, impairment, routing, and measurement pipeline operates over one, four, and nine backbone hops. RTT rose and throughput fell with path length in these controlled scenarios. Second, the complete 50-node/88-link topology was instantiated. Third, the complete 4,224-route plan was generated and validated in dry-run mode.

These are meaningful extensions, but none implies all-pairs evaluation. Real full-topology traffic was limited to three endpoint pairs using 20 selected on-demand routes. The dry-run checks structure and command planning, not runtime route installation, convergence, resource cost, or traffic correctness for every pair. RQ2 is therefore answered within an explicit boundary: the framework scales its topology representation and validates routing logic, and it carries measurable real traffic on representative paths, but exhaustive large-topology traffic coverage remains unevaluated.

6.3 RQ3: AI and learning-based extensibility

The AI result supports the integration mechanism more strongly than any provider-level performance claim. A compatible-provider response passed schema, semantic, prompt, projection, dry-run, and controlled-execution checks, demonstrating that generated scenarios can enter the same guarded pipeline. The official OpenAI request failed because of quota, so provider availability was not established. Separating these outcomes is important for reproducibility: API compatibility is not provider identity, and a successful downstream validator cannot turn a failed upstream call into success.

The RL supplement provides a stronger real-execution test because all 80 valid episodes contain measured Docker traffic. It also produces a negative learning result. The threshold heuristic changed paths at the known phase boundary and maintained high reward with low variance. The evaluated Q-learning policy selected path A throughout; when conditions changed, its rewards became negative and variable. In this low-dimensional, ordered, rule-defined environment, the heuristic encoded the relevant decision boundary directly, whereas 20 episodes, fixed exploration, and the recorded hyperparameters did not produce adaptation. This does not show that RL is generally inferior. It shows that a learnable control loop can function while still losing to a credible simple baseline, which is a more useful conclusion than asserting automation success from implementation alone.

The Dashboard provides access to the small-topology path but is not central evidence for RQ3. Its documented smoke is consistent with the pipeline, yet missing raw validation and regression/security logs prevent a stronger claim. It should be presented as a supporting interface, not as a Germany50 manager or RL training environment.

6.4 Threats to validity and limitations

Internal validity. Experiments share a host, Docker engine, kernel, and virtual networking stack. Uncontrolled host load and scheduling can influence throughput and timing. The retained runners, route/qdisc checks, and repeated cohorts reduce but do not remove this risk. The 100-Mbps cohort contains a preserved failed attempt and retry; inclusion rules use five valid measurements and do not delete the failure.

Construct validity. Configured one-way delay and loss are not the same constructs as measured RTT and end-to-end ping loss. TCP throughput is affected by more than the configured TBF rate. Reward is specific to the retained RL formulation and should not be generalized to all network objectives.

Statistical conclusion validity. Most impairment cohorts contain five runs, and Germany50 extracted paths contain three. Student’s t intervals communicate this uncertainty but are themselves imprecise at small n . The latency cohorts are tightly clustered; packet-loss and 100-Mbps results are more variable. The 20-Mbps, Qwen, and Dashboard observations are single records and do not support interval estimates.

External validity. Results apply to the evaluated Docker/WSL environment, topologies, traffic tools, and impairment ranges. They do not establish equivalence with hardware routers, production workloads, other kernels, or wide-area networks. Germany50 represents structural scale but only three real traffic paths. The Q-learning result applies to one state/action/reward design and one hyperparameter setting.

Reproducibility and evidence preservation. Scenarios, raw metrics, summaries, scripts, plots, failures, and cleanup records provide substantial provenance. Some statistics in this draft are AC derivatives and remain separate from repository evidence. Dashboard raw validation/regression/security logs and dedicated fresh-clone evidence were not retained. These limitations reduce the strength of interface and environment-portability claims but do not invalidate the core impairment, topology, AI-boundary, or RL comparisons.

Overall, the main research question is answered affirmatively within scope. The framework provides a coherent and auditable experimental workflow across controlled conditions and extensions, but its effectiveness is bounded by container-host fidelity, representative rather than exhaustive topology traffic, provider dependence, and the performance of the selected control algorithm.

7 Conclusion

This dissertation evaluated whether a Docker-based NDT can enforce and measure controlled network conditions while remaining reproducible and extensible. For RQ1, repeated bandwidth, delay, and loss experiments produced distinct measured regimes with retained per-run evidence and uncertainty estimates; the results also showed that configuration values must not be conflated with end-to-end TCP, RTT, or loss observations. For RQ2, the same pipeline extended from direct and dual-router topologies to Germany50-derived paths and a complete 50-node/88-link instantiation. Routing claims remain bounded: 4,224 entries were dry-run validated, while real full-topology traffic covered only three representative paths. For RQ3, validated Qwen-compatible generation and 80 real-Docker control episodes demonstrated shared automation hooks, while the official OpenAI call failed with HTTP 429 and the threshold heuristic materially outperformed Q-learning.

The main conclusion is therefore conditional rather than universal. Within the evaluated configurations, the proposed system provides a reproducible workflow for controlled Docker network experimentation, validated routing, structured evidence, and guarded AI/control extensions. Its limitations—shared-host fidelity, small cohorts, representative Germany50 coverage, provider availability, missing Dashboard validation logs, and Q-learning underperformance—define the boundary of the contribution and the most credible directions for future work.

References

- Paul Almasan, Miquel Ferriol-Galmés, Jordi Paillisse, José Suárez-Varela, Diego Perino, Diego López, Antonio Agustin Pastor Perales, Paul Harvey, Laurent Ciavaglia, Leon Wong, Vishnu Ram, Shihan Xiao, Xiang Shi, Xiangle Cheng, Albert Cabellos-Aparicio, and Pere Barlet-Ros. Network digital twin: Context, enabling technologies, and opportunities. *IEEE Communications Magazine*, 60(11):22–27, November 2022. doi: 10.1109/MCOM.001.2200012. URL <https://doi.org/10.1109/MCOM.001.2200012>.
- Nan Cheng, Xiucheng Wang, Zan Li, Zhisheng Yin, Tom H. Luan, and Xuemin Shen. Toward enhanced reinforcement learning-based resource management via digital twin: Opportunities, applications, and challenges. *IEEE Network*, 39(1):189–196, 2025. doi: 10.1109/MNET.2024.3438543. URL <https://doi.org/10.1109/MNET.2024.3438543>.
- Carlos Güemes-Palau, Paul Almasan, Shihan Xiao, Xiangle Cheng, Xiang Shi, Pere Barlet-Ros, and Albert Cabellos-Aparicio. Accelerating deep reinforcement learning for digital twin network optimiza-

- tion with evolutionary strategies. In *NOMS 2022—2022 IEEE/IFIP Network Operations and Management Symposium*, pp. 1–5. IEEE, 2022. doi: 10.1109/NOMS54207.2022.9789945. URL <https://doi.org/10.1109/NOMS54207.2022.9789945>.
- Nikhil Handigol, Brandon Heller, Vimalkumar Jeyakumar, Bob Lantz, and Nick McKeown. Reproducible network experiments using container-based emulation. In *Proceedings of the 8th International Conference on Emerging Networking Experiments and Technologies*, pp. 253–264. Association for Computing Machinery, 2012. doi: 10.1145/2413176.2413206. URL <https://doi.org/10.1145/2413176.2413206>.
- Linbo Hui, Mowei Wang, Liang Zhang, Lu Lu, and Yong Cui. Digital twin for networking: A data-driven performance modeling perspective. *IEEE Network*, 37(3):202–209, May 2023. doi: 10.1109/MNET.119.2200080. URL <https://doi.org/10.1109/MNET.119.2200080>.
- International Telecommunication Union. Digital twin network—requirements and architecture. Recommendation ITU-T Y.3090, ITU Telecommunication Standardization Sector, February 2022. URL <https://www.itu.int/rec/T-REC-Y.3090/en>.
- iproute2 Project. *tc-netem(8): Network Emulator*, n.d. URL <https://man7.org/linux/man-pages/man8/tc-netem.8.html>. Linux manual page; accessed 2026-08-10.
- Simon Knight, Hung X. Nguyen, Nickolas Falkner, Rhys Bowden, and Matthew Roughan. The internet topology zoo. *IEEE Journal on Selected Areas in Communications*, 29(9):1765–1775, October 2011. doi: 10.1109/JSAC.2011.111002. URL <https://doi.org/10.1109/JSAC.2011.111002>.
- Bob Lantz, Brandon Heller, and Nick McKeown. A network in a laptop: Rapid prototyping for software-defined networks. In *Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks*, pp. 1–6. Association for Computing Machinery, 2010. doi: 10.1145/1868447.1868466. URL <https://doi.org/10.1145/1868447.1868466>.
- Oscar G. Lira, Oscar M. Caicedo, and Nelson L. S. da Fonseca. Large language models for zero touch network configuration management. *IEEE Communications Magazine*, 63(7):146–153, 2025. doi: 10.1109/MCOM.001.2400368. URL <https://doi.org/10.1109/MCOM.001.2400368>.
- Chang Liu, Xiaohui Xie, Xinggong Zhang, and Yong Cui. Large language models for networking: Workflow, advances and challenges. *IEEE Network*, 39(5):165–172, September 2025. doi: 10.1109/MNET.2024.3510936. URL <https://doi.org/10.1109/MNET.2024.3510936>.
- Hongzi Mao, Mohammad Alizadeh, Ishai Menache, and Srikanth Kandula. Resource management with deep reinforcement learning. In *Proceedings of the 15th ACM Workshop on Hot Topics in Networks*, pp. 50–56. Association for Computing Machinery, 2016. doi: 10.1145/3005745.3005750. URL <https://doi.org/10.1145/3005745.3005750>.
- Dirk Merkel. Docker: Lightweight linux containers for consistent development and deployment. *Linux Journal*, 2014. URL <https://www.linuxjournal.com/content/docker-lightweight-linux-containers-consistent-development-and-deployment>.
- Manuel Peuster, Holger Karl, and Steven van Rossem. MeDICINE: Rapid prototyping of production-ready network services in multi-PoP environments. In *2016 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN)*, pp. 148–153. IEEE, 2016. doi: 10.1109/NFV-SDN.2016.7919490. URL <https://doi.org/10.1109/NFV-SDN.2016.7919490>.
- Cheng Zhou, Hongwei Yang, Xiaodong Duan, Diego Lopez, Antonio Pastor, Qin Wu, Mohamed Boucadair, and Christian Jacquenet. Network digital twin: Concepts and reference architecture. Internet-Draft draft-irtf-nmrg-network-digital-twin-arch-12, Internet Research Task Force, February 2026. URL <https://datatracker.ietf.org/doc/draft-irtf-nmrg-network-digital-twin-arch/12/>. Work in Progress.

A Appendix

A.1 Detailed experiment and provenance notes

Table 4 expands the main experimental matrix with evidence locations. Paths are relative to the read-only evidence repository unless prefixed by **derived/**, which denotes a dissertation-side audit derivative.

Table 4: Detailed evidence index for the principal experiments.

Experiment	Raw or primary evidence	Summary / reproduction
Historical BW 20	runs/current/metrics.json	scripts/simulator_routed.py
BW 50/100	runs/bandwidth-evidence-supplement/ replication-attempt-*	summary.json, per-run-results.csv, scripts/run_ bandwidth_evidence_supplement.py
Latency	runs/delay_batch_20260713-135915/delay_*_run_ */metrics.json	runs/final-evaluation/evidence-inventory.json, scripts/run_delay_batch.py
Packet loss	runs/packet_loss_batch_20260713-153303/loss_ *_run_*/metrics.json	runs/final-evaluation/evidence-inventory.json, scripts/run_packet_loss_batch.py
Dual-router	runs/two_router_batch_20260713-160810/ baseline_run_*/metrics.json	scripts/run_two_router_batch.py
Germany50 paths	runs/germany50-selected-paths-final/*-run-*/ metrics.json	summary.json, summary.csv, scripts/run_germany50_ selected_path_tests.py
Germany50 full	runs/final-evaluation/germany50-full-fixed/ selected-attempt-1/	route-plan-dry-run/summary.json, scripts/run_ germany50_full_evaluation.py
Official OpenAI	Sanitized source reference only	runs/final-evaluation/ai-openai-429-evidence. json
Qwen-compatible	runs/openai-compatible-live-validation- 20260714-064525/	runs/final-evaluation/ai-qwen-live-summary.json
RL	runs/final-evaluation/rl-docker-supplement/ per-episode-results.csv	summary.json, policy-summary.csv, scripts/run_rl_ docker_supplement.py
Dashboard	Raw validation logs not retained	docs/final/interactive-dashboard-validation.md

The dissertation-side consolidated inventory is `notes/experiment_inventory.md`. Its machine-readable counterpart is `derived/audit_computed_statistics/experiment_summary.csv`. RR values reproduce repository summaries; AC values were computed from retained per-run data without changing the evidence repository.

A.2 Configuration and measurement details

The direct supplementary bandwidth cohorts used reverse TCP iperf3 for five seconds with server-side TBF. The 50-Mbps cohort has five valid attempts. The 100-Mbps result uses five valid measurements from six planned attempts; the failed record and run-06 retry are retained. The historical routed 20-Mbps value is one record only.

Latency used configured one-way delays of 0/10/30/50 ms with five runs and three ping packets per run. Packet loss used configured one-way probabilities of 0/1/3/5% with five runs and 100 packets per run. For both experiments, measured end-to-end quantities are reported separately from configured one-way impairments.

A.3 Germany50 selected paths

Table 5: Germany50 extracted-path configurations and audit statistics.

Class	Backbone nodes	Hops	RTT mean	RTT SD	Rate mean
Shortest	Aachen-Koeln	1	41.196	0.259	18.8
Median	Erfurt-Wuerzburg-Augsburg-Muenchen-Passau	4	104.623	0.845	17.8
Longest	Oldenburg-Bremen-Hannover-Braunschweig- Kassel-Erfurt-Wuerzburg-Augsburg-Muenchen- Passau	9	214.425	0.441	15.7

Each extracted link used 20 Mbps, 10-ms configured one-way delay, and 0% configured loss. These are path-extracted experiments, not full-topology deployments. The separate full-topology selected-mode run installed 20 routes for the three endpoint pairs. The 4,224-entry plan was dry-run validated only.

A.4 RL policy settings and complete policy summary

The Q-learning run recorded seed 20260722, $\alpha = 0.35$, $\gamma = 0.85$, and $\epsilon = 0.18$. Each policy has 20 valid episodes under the same ordered two-phase conditions. Failed/retried checkpoints remain in the evidence and are excluded from valid-row statistics.

Table 6: Repository-reported real-Docker policy summary.

Policy	n	Reward mean	Reward SD	RTT (ms)	Loss (%)	Rate (Mbps)
Q-learning	20	4.143	15.045	89.225	2.500	9.855
Threshold heuristic	20	17.116	0.325	30.189	0.000	18.625
Fixed A	20	-1.419	21.126	99.609	12.500	9.812
Fixed B	20	0.257	19.665	91.027	10.000	9.808

The reward CI half-widths were 6.594, 0.142, 9.259, and 8.619 in the row order above. The result supports a functioning measured control loop and heuristic superiority in this setup; it does not establish universal RL inferiority.

A.5 AI and Dashboard boundaries

The AI execution chain comprises structured generation, schema validation, semantic validation, prompt-constraint checking, projection to an executable scenario, dry-run, and controlled Docker execution. Official OpenAI returned HTTP 429 `insufficient_quota`; a separate Qwen-compatible provider completed validation. The provider distinction must be preserved in any extracted result or presentation.

The Dashboard supports bounded small-topology interaction. Its documented run `3cada000bb4d` used 20 Mbps, zero delay/loss, three pings, and three seconds of `iperf3`. Documentation reports 3/3 replies, 0.101/0.118/0.143-ms RTT, 19.4 Mbps, successful downloads, and cleanup. Raw UI-path, 69-test, security, secret-scan, and fresh-clone validation logs were not retained, so these checks are not presented as independently auditable evidence.

A.6 Reproduction boundary

The repository includes scenarios, scripts, evidence summaries, and a reproducibility guide. Re-running the formal matrices is neither necessary nor part of this dissertation preparation workflow. A future independent replication should record host/kernel/Docker versions, image digests, complete stdout/stderr, resource utilization, and fresh-clone validation, then compare its new results without overwriting the sealed evidence used here.